Builder Pattern

Category: Creational Design Pattern

1. The Problem

When an object requires many parameters — especially optional ones — a single constructor can become hard to use or maintain.

```
        public class House {
            private String foundation;
            private String structure;
            private String roof;
            private boolean hasGarage;
            private boolean hasSwimmingPool;
            private boolean hasGarden;

            // Constructor
            public House(String foundation, String structure, String roof,
                         boolean hasGarage, boolean hasSwimmingPool, boolean hasGarden){
                this.foundation = foundation;
                this.structure = structure;
                this.roof = roof;
                this.hasGarage = hasGarage;
                this.hasSwimmingPool = false;
                this.hasGarden = hasGarden;
            }

            @Override
            public String toString() {
                return "House{" +
                        "foundation='" + foundation + '\'' +
                        ", structure='" + structure + '\'' +
                        ", roof='" + roof + '\'' +
                        ", hasGarage=" + hasGarage +
                        ", hasSwimmingPool=" + hasSwimmingPool +
                        ", hasGarden=" + hasGarden +
                        '}';
            }
        }

        public class WithoutBuilderPattern {
            public static void main(String[] args) {
                House house = new House("Concrete", "Wood", "Shingles", true, true, false);
                // Constructor Explosion -> Too Many Constructors
                // Difficult to understand and maintain this code
                System.out.println(house);
            }
        }
```

Issues: - **Long constructor parameter lists** — as more fields (especially optional ones) are added, the constructor keeps growing. - **Hard to tell which values are optional vs. required** — a call like `new House("Concrete", "Wood", "Shingles", true, true, false)` gives no clue at the call site what each `boolean` actually means. - **No flexibility to set only some values** — to change one optional field,

every other parameter still has to be supplied in the exact right order, and adding overloaded constructors for different combinations leads to “constructor explosion.”

2. The Solution — Builder Pattern

Intent: Separate the construction of a complex object from its final representation, allowing the same construction process to build it up step by step — typically through a fluent, chainable interface.

- A nested `HouseBuilder` class collects the required parameters via its own constructor, and exposes chainable setter methods for optional ones.
- Calling `.build()` at the end assembles and returns the fully constructed, immutable `House`.

3. Key Idea

- **Decouple “what to build” from “how it’s assembled”** — the client describes what it wants (mandatory + selected optional fields) through method calls, and the builder handles turning that into a valid object.
- **Fluent interface** — each setter returns `this` (the builder itself), allowing calls to be chained: `.setGarden(true).setGarage(true).build()`.
- **Immutability of the built object** — `House`’s constructor is `private` and only accessible to `HouseBuilder`, so once built, a `House`’s fields can’t be reassigned from outside.
- Required fields are enforced through the builder’s own constructor; optional fields default sensibly and are only set if the client explicitly calls the corresponding method.

4. Variants

- **Static nested Builder class** (shown here) — the builder lives inside the class it builds (`House.HouseBuilder`), which is the most common Java implementation.
- **Standalone/separate Builder class** — the builder can also live as its own top-level class implementing a common `Builder` interface, useful when multiple product variations need different builder implementations (this shades into the Abstract Factory-style “Director + Builder” GoF structure, where a separate `Director` class controls the build sequence).
- **Telescoping constructors** (the “without pattern” approach, using multiple overloaded constructors) is sometimes mentioned as the naive alternative the Builder Pattern replaces — it doesn’t scale well as optional parameters grow.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Product	The complex object being constructed	`House`
Builder	Collects construction parameters (mandatory via constructor, optional via chainable setters) and assembles the final product	`HouseBuilder`
Client	Uses the builder’s fluent interface to construct the product	`BuilderPattern` (main method)

Relationship type: Composition/aggregation — `HouseBuilder` is a **static nested class** inside `House`; the finished `House` object is built *from* the builder's accumulated state, and the builder holds a full copy of all the field values until `build()` is called.

6. Code — Full Implementation

```
public class House {
    private String foundation;
    private String structure;
    private String roof;
    private boolean hasGarage;
    private boolean hasSwimmingPool;
    private boolean hasGarden;

    // Constructor
    private House(HouseBuilder builder) {
        this.foundation = builder.foundation;
        this.structure = builder.structure;
        this.roof = builder.roof;
        this.hasGarage = builder.hasGarage;
        this.hasSwimmingPool = builder.hasSwimmingPool;
        this.hasGarden = builder.hasGarden;
    }

    @Override
    public String toString() {
        return "House{" +
                "foundation='" + foundation + '\'' +
                ", structure='" + structure + '\'' +
                ", roof='" + roof + '\'' +
                ", hasGarage=" + hasGarage +
                ", hasSwimmingPool=" + hasSwimmingPool +
                ", hasGarden=" + hasGarden +
                '}';
    }

    public static class HouseBuilder {
        private String foundation;
        private String structure;
        private String roof;
        private boolean hasGarage;
        private boolean hasSwimmingPool;
        private boolean hasGarden;

        // Builder constructor with mandatory parameters
        public HouseBuilder(String foundation, String structure, String roof){
            this.foundation = foundation;
            this.structure = structure;
            this.roof = roof;
        }

        // Optional parameters
        public HouseBuilder setGarden(boolean hasGarden){
            this.hasGarden = hasGarden;
            return this;
        }
        public HouseBuilder setSwimmingPool(boolean hasSwimmingPool){
            this.hasSwimmingPool = hasSwimmingPool;
            return this;
```

```
            }
            public HouseBuilder setGarage(boolean hasGarage){
                this.hasGarage = hasGarage;
                return this;
            }
            public House build(){
                return new House(this);
            }
        }
    }

    public class BuilderPattern {
        public static void main(String[] args) {

            House house = new House.HouseBuilder("Concrete", "Wooden", "Tile")
                    .setGarden(true)
                    .build();

            System.out.println(house);
        }
    }
```

What changed vs. the "without pattern" version: - One large public constructor with 6 positional parameters → a `private` `House` constructor that only accepts a `HouseBuilder`, plus a small `HouseBuilder` constructor for just the *mandatory* fields. - All 6 fields had to be supplied at once, in order → optional fields (`hasGarage`, `hasSwimmingPool`, `hasGarden`) are now set individually via named, chainable methods, only when needed. - No way to tell what a positional `true` / `false` argument meant at the call site → `setGarden(true)` is self-documenting. - Object assembly happens in a single constructor call → assembly now happens step by step through the builder, finished off by `.build()`.

7. Benefits

- **Readability** — `.setGarden(true).setGarage(true).build()` is self-documenting compared to a long list of positional arguments.
- **Flexibility** — only the optional fields that are actually needed have to be set; everything else keeps its default.
- **Avoids constructor explosion** — no need for many overloaded constructors to cover different combinations of optional parameters.
- **Immutability** — the final product (`House`) can be made immutable, since its fields are only ever set once, through the builder, at construction time.
- **Encapsulated construction logic** — validation or defaulting logic for how a valid object is assembled can live entirely inside the builder.

8. Drawbacks / Things to Watch Out For

- **More code** — introducing a builder (often a nested static class) adds boilerplate compared to a plain constructor, especially for simple objects with few fields.
- **Overkill for simple objects** — if a class has only 1–2 optional fields, a builder may be unnecessary complexity; a few overloaded constructors or default parameter values might suffice.
- **Mandatory fields still need discipline** — if mandatory fields aren't enforced through the builder's own constructor (as done here), it's possible to build an incomplete/invalid object by mistake.

9. Real-World Use Cases

- **Object construction with many optional configuration fields** — e.g. building HTTP request objects, UI components, or configuration objects with many optional settings.
- **String/Query builders** — e.g. Java's `StringBuilder`, or SQL query builders that assemble a query clause by clause.
- **Immutable data classes** — building complex immutable objects (like DTOs) where all fields must be set at construction and never changed afterward.
- **Document/Report generation** — building up a complex document (e.g. a PDF or HTML report) section by section before finalizing it.

10. Interview Questions & Answers

Q1. What is the Builder Pattern? A creational design pattern that separates the construction of a complex object from its final representation, allowing the object to be built step by step — typically via a fluent, chainable interface — instead of through one large constructor.

Q2. What problem does it solve? It solves the "constructor explosion" / telescoping constructor problem, where a class with many parameters — especially optional ones — becomes hard to construct correctly and hard to read at the call site, since positional arguments don't convey what they represent.

Q3. What are its main components/participants? - **Product** — the complex object being built (`House`) - **Builder** — collects parameters and assembles the product, usually via chainable setters (`HouseBuilder`) - **Client** — drives construction through the builder's fluent interface

Q4. Why is the `House` constructor made `private` in this example? To force all construction to go through `HouseBuilder` — since `House`'s only constructor takes a `HouseBuilder` and is private, there is no way to create a `House` object except by using the builder, guaranteeing the built object is always assembled consistently.

Q5. How does the Builder Pattern achieve a "fluent interface"? Each optional setter method (e.g. `setGarden()`, `setGarage()`) returns `this` — the builder instance itself — instead of `void`. This allows successive method calls to be chained directly onto one another: `new HouseBuilder(...).setGarden(true).setGarage(true).build()`.

Q6. How does the Builder Pattern differ from the Factory Pattern? Both are creational patterns, but they solve different problems: **Factory** is about deciding *which concrete class* to instantiate (often based on some input/type), typically returning the object in a single call. **Builder** is about constructing *one specific, complex object* step by step, especially when it has many optional parts — the focus is on the assembly process, not on choosing between different types.

Q7. What are the benefits of the Builder Pattern? Improved readability at the call site, flexibility to set only the needed optional fields, avoidance of constructor explosion, support for building immutable objects, and encapsulation of construction/validation logic inside the builder.

Q8. What are the drawbacks? Extra boilerplate code compared to a plain constructor, which can be overkill for simple objects with few fields, and the risk of building invalid/incomplete objects if mandatory fields aren't properly enforced by the builder itself.

Q9. Can you give real-world examples? Java's `StringBuilder`, SQL query builders, HTTP request builders, UI component builders with many optional configuration options, and step-by-step construction of immutable data objects or documents.